

Exercices – Fonctions, collections, fichiers et générateurs (Python)

Exercice 1 – Transformation avec list comprehension

Écrire une fonction `square_even(numbers)` qui :

- reçoit une liste de nombres
- retourne une nouvelle liste contenant le carré des nombres pairs uniquement

Contrainte :

- utiliser une list comprehension
-

Exercice 2 – Filtrage de dictionnaire

Étant donné le dictionnaire suivant :

```
students = {  
    "Alice": 75,  
    "Bob": 58,  
    "Charlie": 92,  
    "Diana": 64  
}
```

Écrire une fonction qui :

- retourne un nouveau dictionnaire contenant uniquement les étudiants ayant une note supérieure ou égale à 60
-

Exercice 3 – Manipulation de tuples

Étant donné la liste de points suivante :

```
points = [(2, 3), (5, 1), (0, 4), (3, 3)]
```

Écrire une fonction qui :

- retourne le point le plus éloigné de l'origine `(0, 0)`

Indice :

- utiliser les tuples sans les modifier
-

Exercice 4 – Lecture depuis un fichier

Écrire une fonction `read_numbers(filename)` qui :

- lit un fichier texte contenant un nombre par ligne
- retourne une liste d'entiers

Hypothèse :

- le fichier est bien formaté
-

Exercice 5 – Écriture vers un fichier

À partir de la liste suivante :

```
values = [3, 7, 2, 9, 4]
```

Écrire une fonction qui :

- trie les nombres
 - écrit les valeurs dans un fichier texte, un nombre par ligne
-

Exercice 6 – Générateur simple

Écrire un générateur `countdown(n)` qui :

- génère les valeurs de `n` jusqu'à `0`
- retourne une valeur à la fois

Utiliser :

- l'instruction `yield`
-

Exercice 7 – Générateur conditionnel

Écrire un générateur `even_numbers(numbers)` qui :

- reçoit une liste de nombres
- génère uniquement les nombres pairs

Ajouter un court commentaire comparant cette approche à une list comprehension.

Exercice 8 – Dictionnaire inversé

Étant donné le dictionnaire suivant :

```
grades = {  
    "Alice": "A",  
    "Bob": "B",  
    "Charlie": "A",  
    "Diana": "C"  
}
```

Créer un nouveau dictionnaire qui :

- regroupe les étudiants par note

Résultat attendu :

```
{  
    "A": ["Alice", "Charlie"],  
    "B": ["Bob"],  
    "C": ["Diana"]  
}
```

Exercice 9 – Tuple comme clé de dictionnaire

Étant donné la liste suivante :

```
coordinates = [(1, 2), (3, 4), (1, 2), (5, 6)]
```

Créer un dictionnaire qui :

- compte le nombre d'occurrences de chaque coordonnée

Indice :

- utiliser les tuples comme clés
-

Exercice 10 – Pipeline de traitement de données

Écrire une fonction qui :

1. lit des nombres depuis un fichier
2. filtre les nombres positifs
3. génère le carré de ces nombres
4. écrit le résultat dans un autre fichier

Contraintes :

- utiliser au moins une fonction
- utiliser un générateur ou une list comprehension
- le code doit être clair et bien structuré